

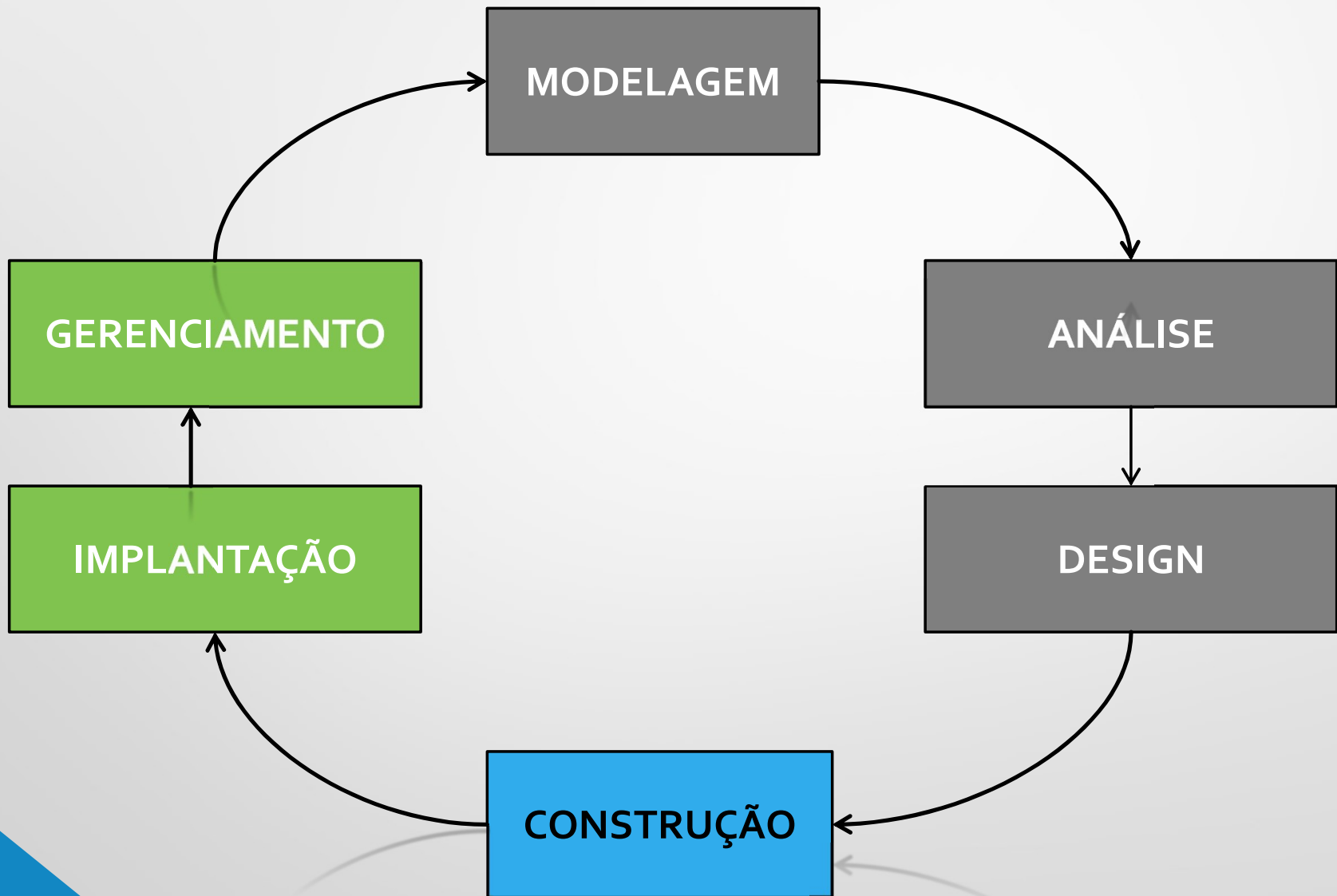


Tópico 3.4:

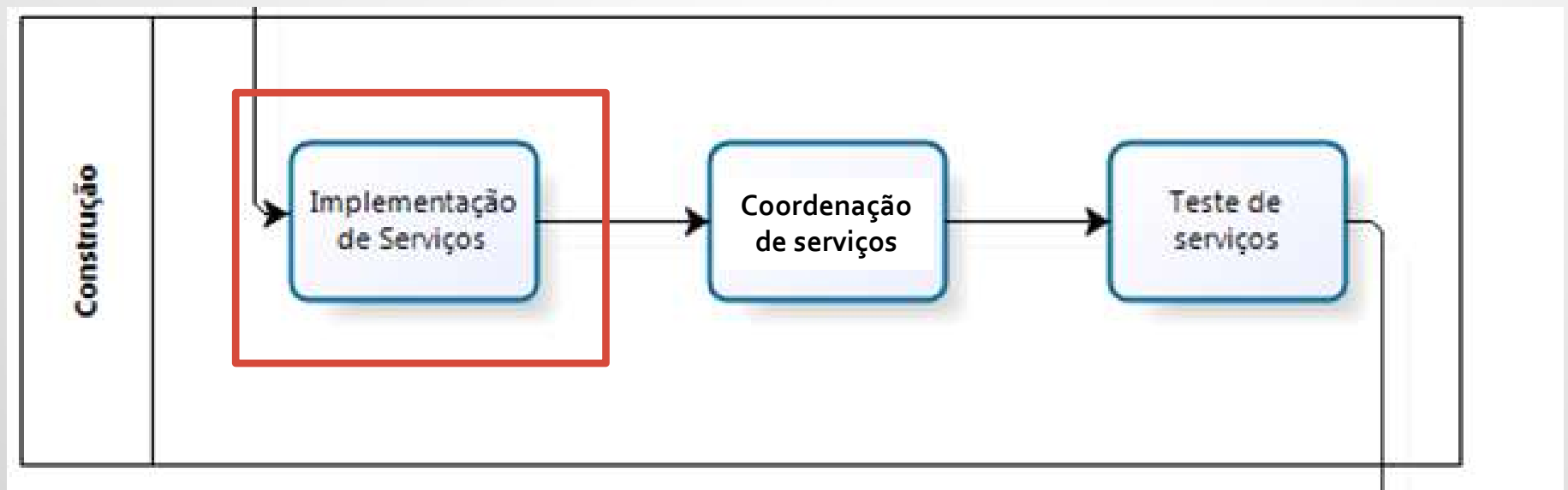
Engenharia de Software Orientada a Serviços

Fase de Construção

Fases do Ciclo de Vida SOA



Construção



Construção - Implementação

Implementação de Componentes de Serviços:

É a atividade responsável por **desenvolver tecnicamente os serviços**, conforme os contratos definidos na fase de design.

Implementação de Serviços

- Classes de Mensagens (DTOs)
- Classes de Fachada (Interfaces / Endpoints)
- Classes de Negócio
- Classes de Acesso a Dados

Construção - Implementação

Implementação de Componentes de Serviços:

➤ Componentes de Infraestrutura

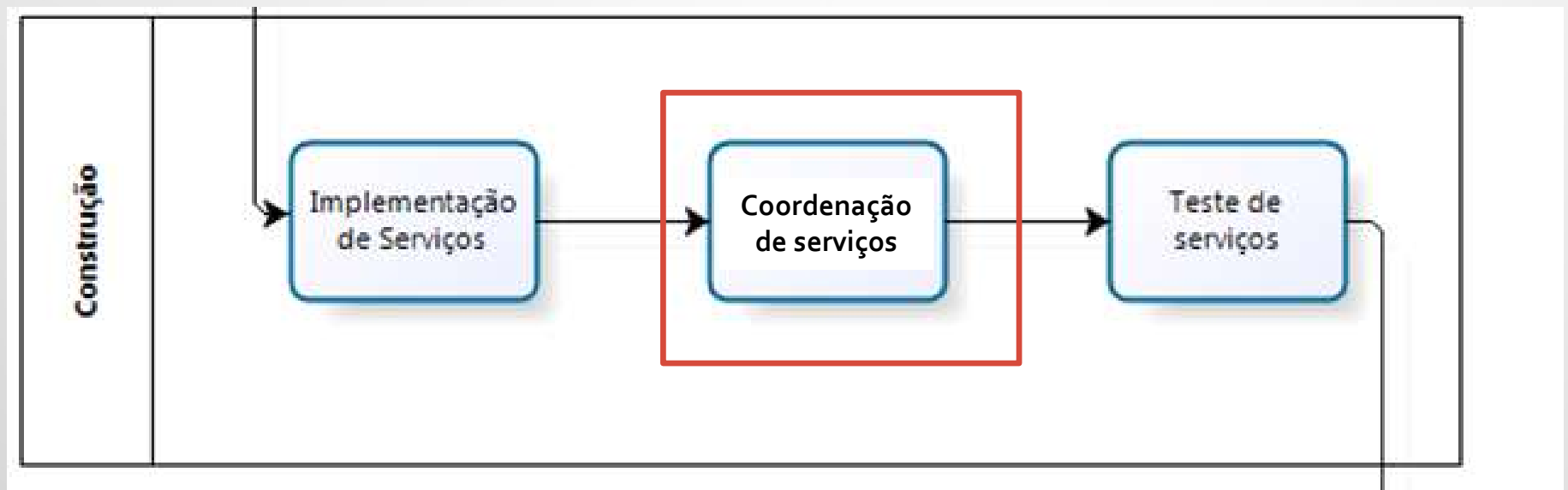
- Comunicação (REST/SOAP)
- Segurança
- **Adapters / Integradores (APIs externas)**

Construção

Componente vs Serviço

Aspecto	Componentes	Serviços
Escopo	Interno ao sistema	Externo (entre sistemas)
Comunicação	Chamada de método (direta)	HTTP, REST, mensagens
Acoplamento	Médio (dependem diretamente)	Baixo (contratos bem definidos)
Interface	Interfaces fornecidas/requeridas	Contrato de serviço (JSON, API)
Exemplo	ClienteComponent.buscarCliente()	GET /clientes/{id}

Construção



Coordenação de Serviços

É a atividade responsável por **orquestrar** ou **coreografar** a interação entre múltiplos serviços para executar um processo de negócio.



Orquestração

VS



Coreografia

Orquestração de Serviços

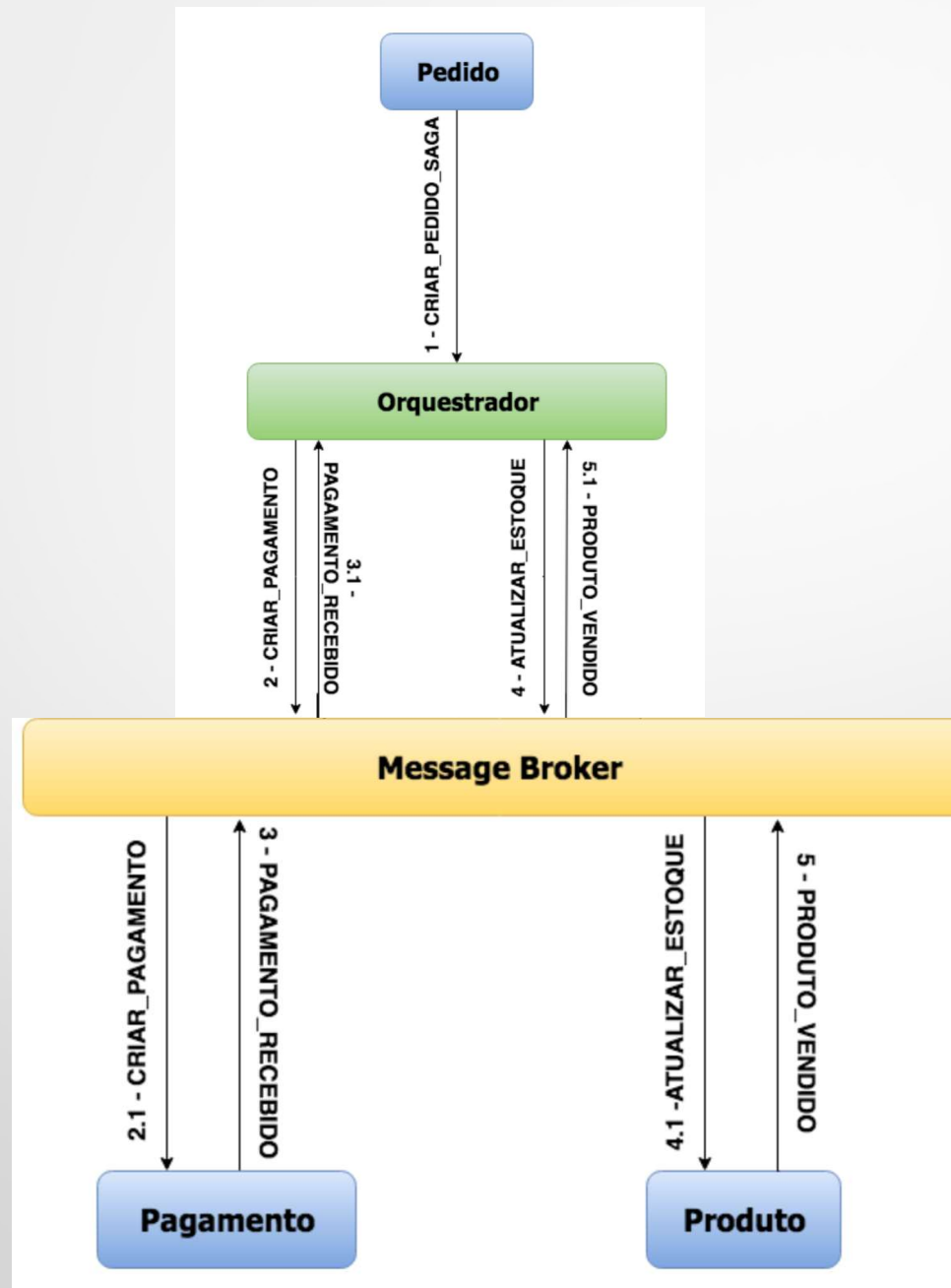
- Um modelo de composição onde um processo central controla explicitamente a execução dos serviços.
- Características
 - ✓ Controle centralizado (um “orquestrador”)
 - ✓ Fluxo de execução explícito
 - ✓ Baseado em processos de negócio
 - ✓ Fácil de monitorar e auditar
 - ✓ Forte alinhamento com workflows

Orquestração de Serviços

➤ Papel do Orquestrador

- ✓ Invocar serviços
- ✓ Controlar ordem de execução
- ✓ Tratar exceções
- ✓ Gerenciar estado do processo

Coordenação - Orquestração



Coreografia de Serviços

- Um modelo onde **não existe controle central**, e os serviços interagem por meio de troca de mensagens.
- Características
 - ✓ Descentralização total
 - ✓ Cada serviço conhece apenas sua parte
 - ✓ Comunicação baseada em mensagens/eventos
 - ✓ Alto desacoplamento
 - ✓ Difícil de visualizar o fluxo global

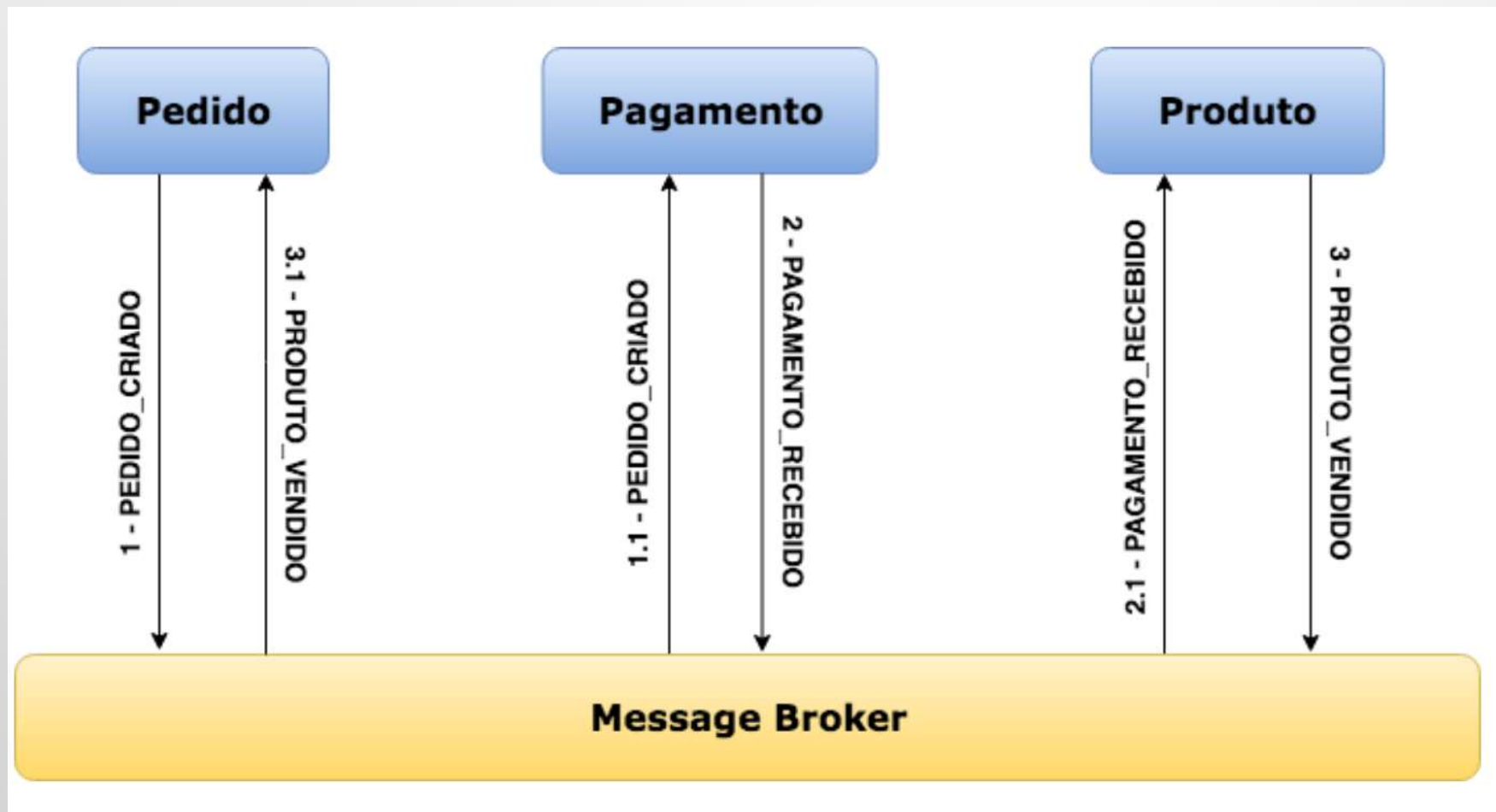
Coreografia de Serviços

➤ Como funciona:

- ✓ Um serviço executa uma ação
- ✓ Emite uma mensagem/evento
- ✓ Outros serviços reagem

Não existe “processo central”

Coordenação - Coreografia



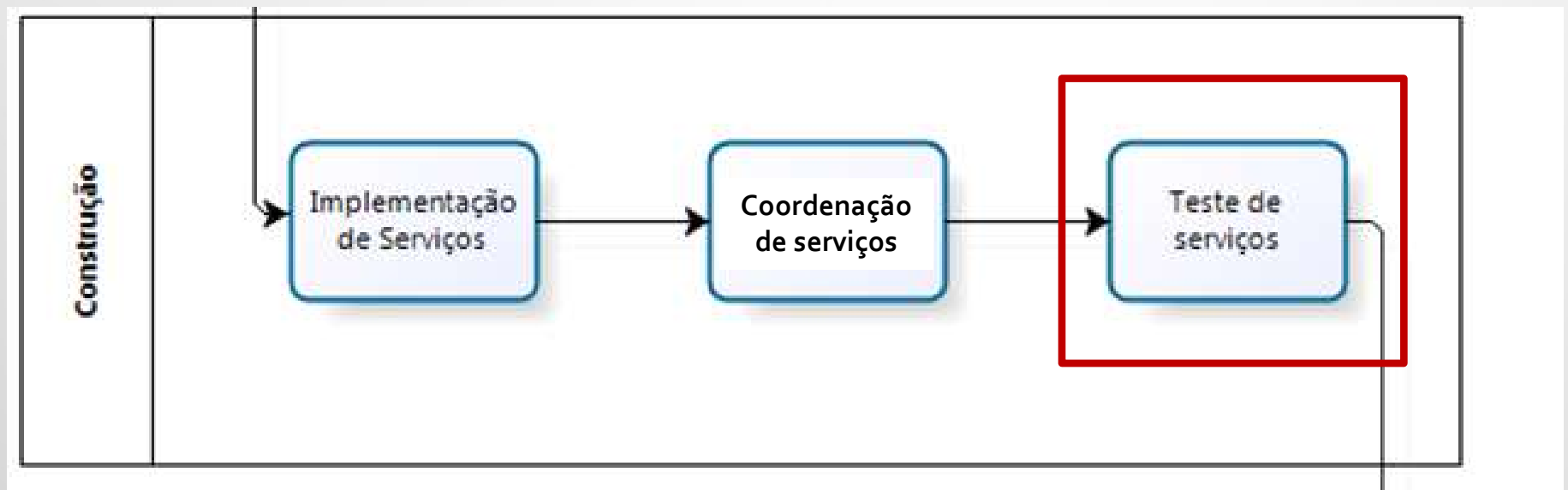
Coordenação de Serviços

- A escolha entre **coreografia** e **orquestração** depende das características e dos requisitos específicos de cada sistema.
- Ao desenhar sua arquitetura, considere os seguintes fatores:
 - Se o fluxo precisa ser **autônomo e resiliente**, a coreografia pode ser a melhor escolha.
 - Se você precisa de **controle centralizado e visibilidade clara**, a orquestração provavelmente atenderá melhor suas necessidades.

Coordenação de Serviços

Aspecto	Orquestração	Coreografia
Facilidade de alteração	Fácil alterar o fluxo alterando o orquestrador.	Alterações são mais complexas: cada serviço pode precisar ser ajustado.
Resiliência	Mais vulnerável: se o orquestrador falhar, tudo pode parar.	Mais resiliente: sem ponto único de falha.
Complexidade	Complexidade no orquestrador.	Complexidade distribuída entre os serviços.
Rastreabilidade	Com um ponto de controle central, identificar e resolver problemas se torna mais fácil.	Localizar a origem de falhas pode ser complicado, pois a lógica do fluxo está dispersa entre serviços.

Construção

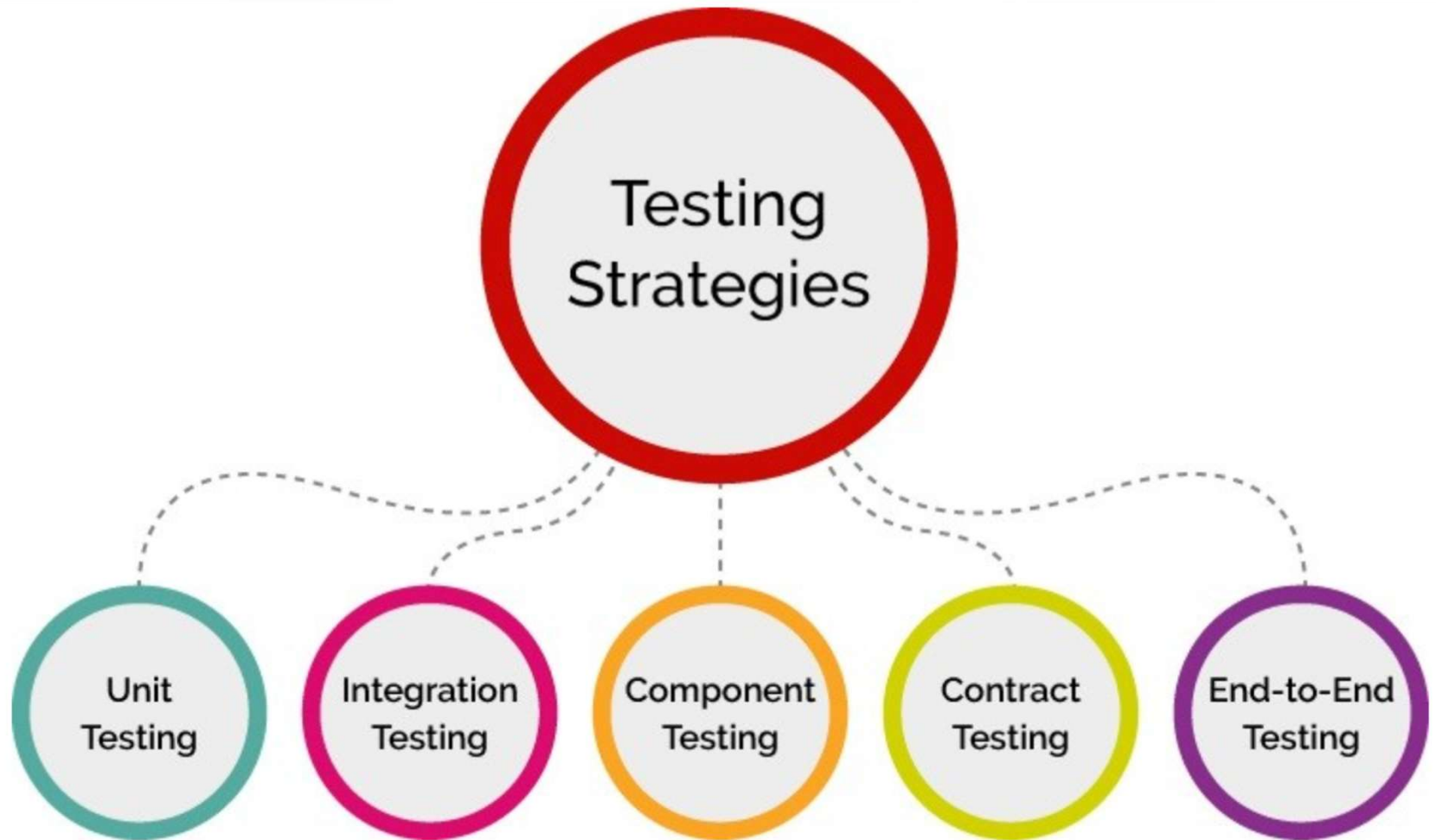


Construção - Teste

Teste:

- É a atividade responsável por validar o funcionamento dos serviços individualmente e em conjunto, garantindo que atendam aos contratos definidos.

Construção - Teste

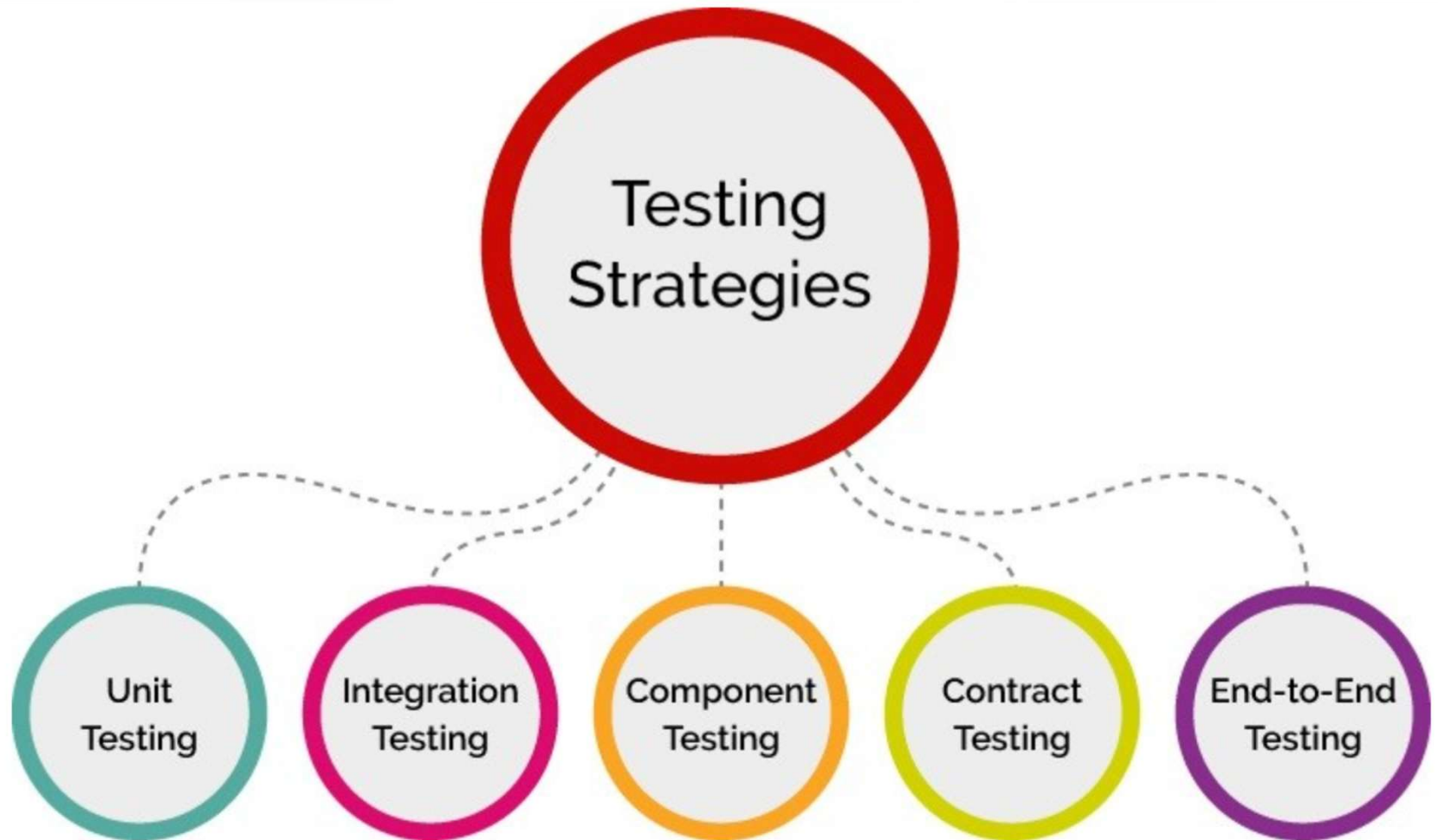


Construção – Teste

1. Testes unitários:

- Verificar o funcionamento correto de unidades isoladas.
- Testam operações isoladas do serviço.
- Ex: Serviço Paciente
 - ✓ Validar funções como buscar paciente pelo nome ou inserir paciente

Construção - Teste

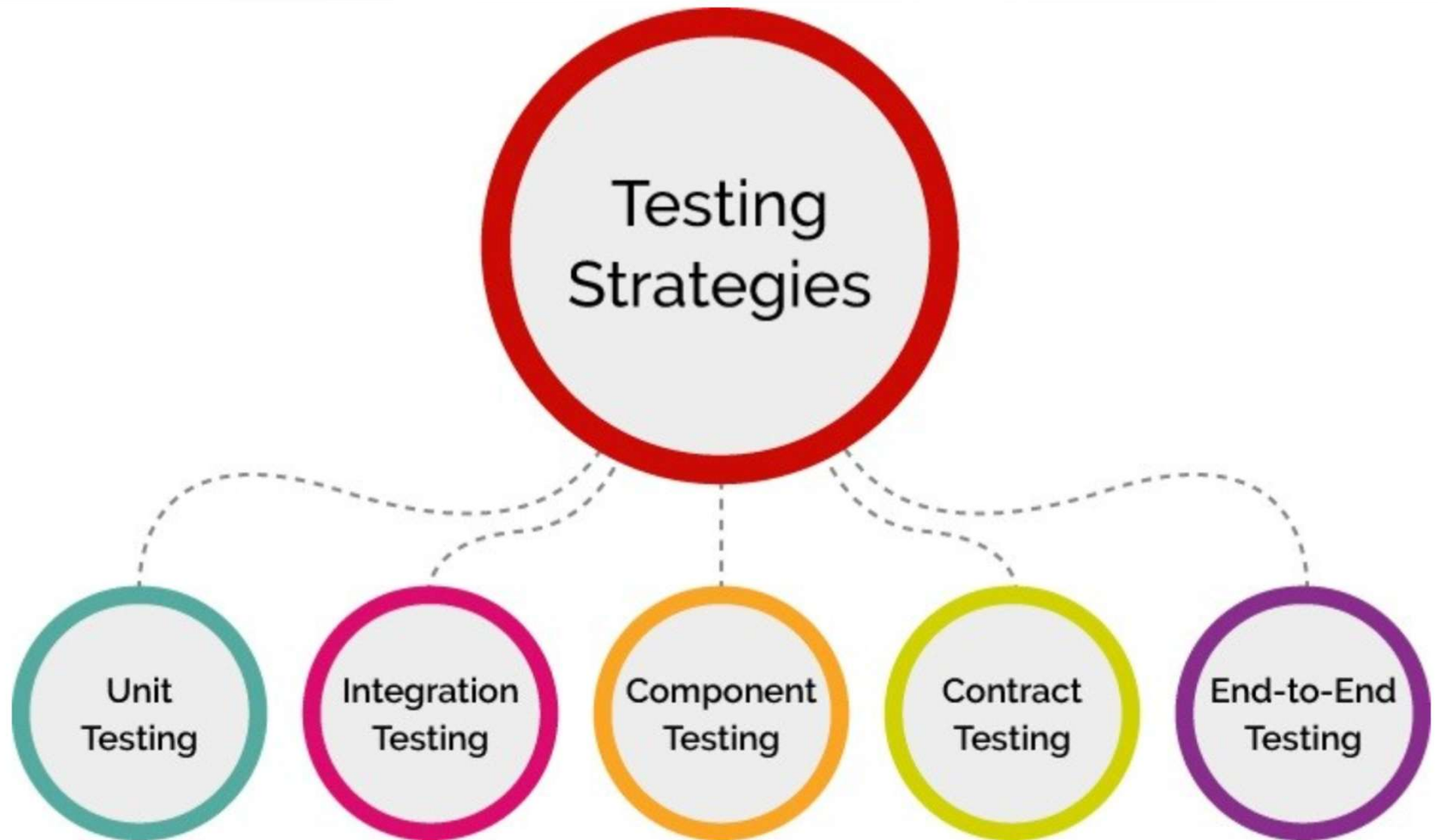


Construção – Teste

2. Testes de integração:

- Utilizados para testar a comunicação entre os serviços e sistemas externos como BD.
- Ex: Testar se o sistema está persistindo corretamente os dados no banco de dados
- Ex: Testar se o paciente recebeu uma notificação de consulta marcada.

Construção - Teste

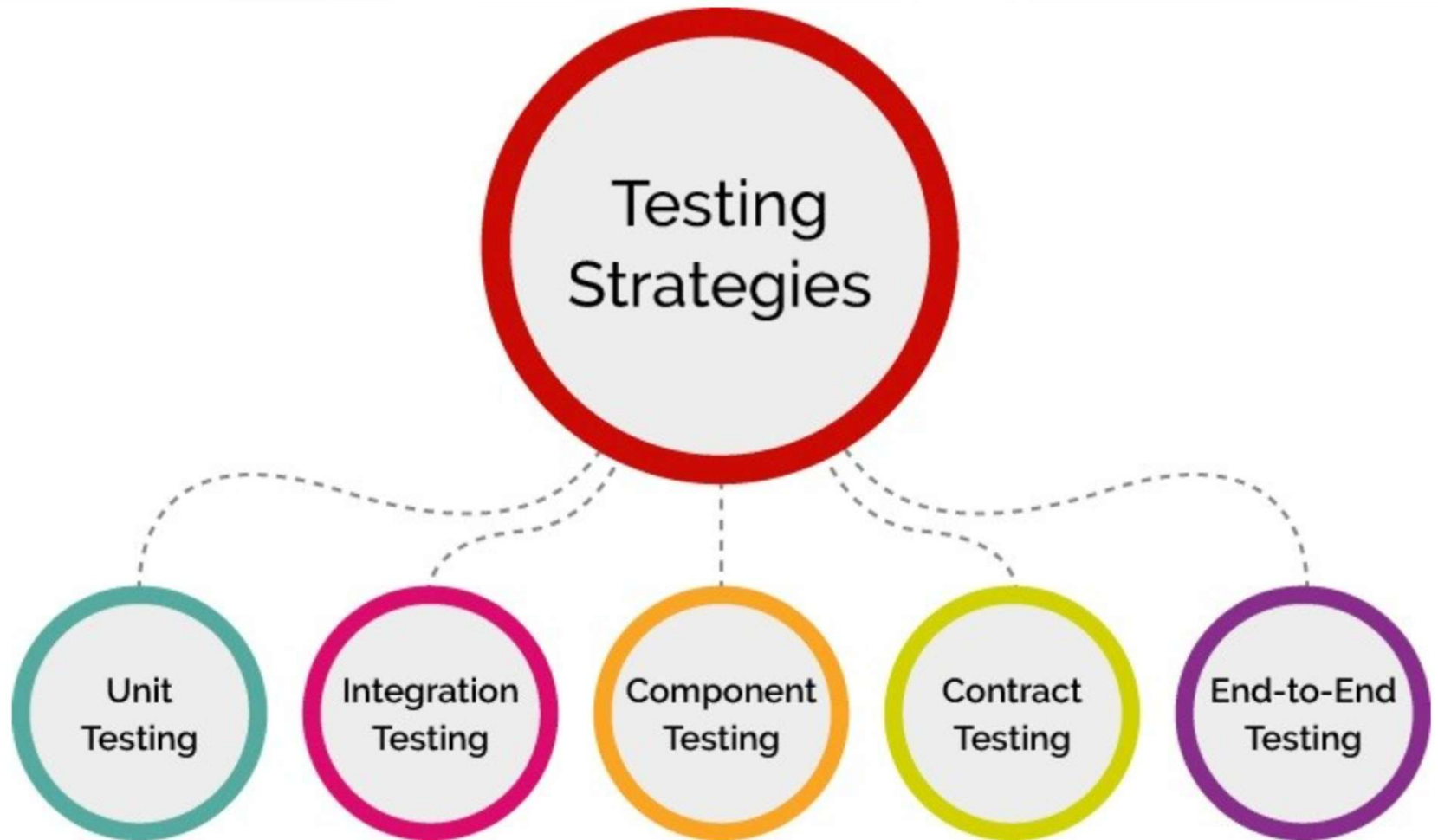


Construção – Teste

3. Testes de componente:

- Verifica se o serviço faz o que deveria fazer.
- Utilizados para testar as funcionalidades completas de um único serviço.
- Validam como partes interdependentes, podem funcionar caso sejam chamados de forma isolada ou integrados a outros serviços.

Construção - Teste



Construção - Teste

4. Testes de contrato de serviço:

- Verifica se a interface esperada entre cliente e servidor está correta.
- Foca na estrutura do request/response (ex: campos obrigatórios, tipos de dados).
- Testa o serviço como uma caixa preta, reproduzindo o ponto de vista do consumidor.

Construção - Teste

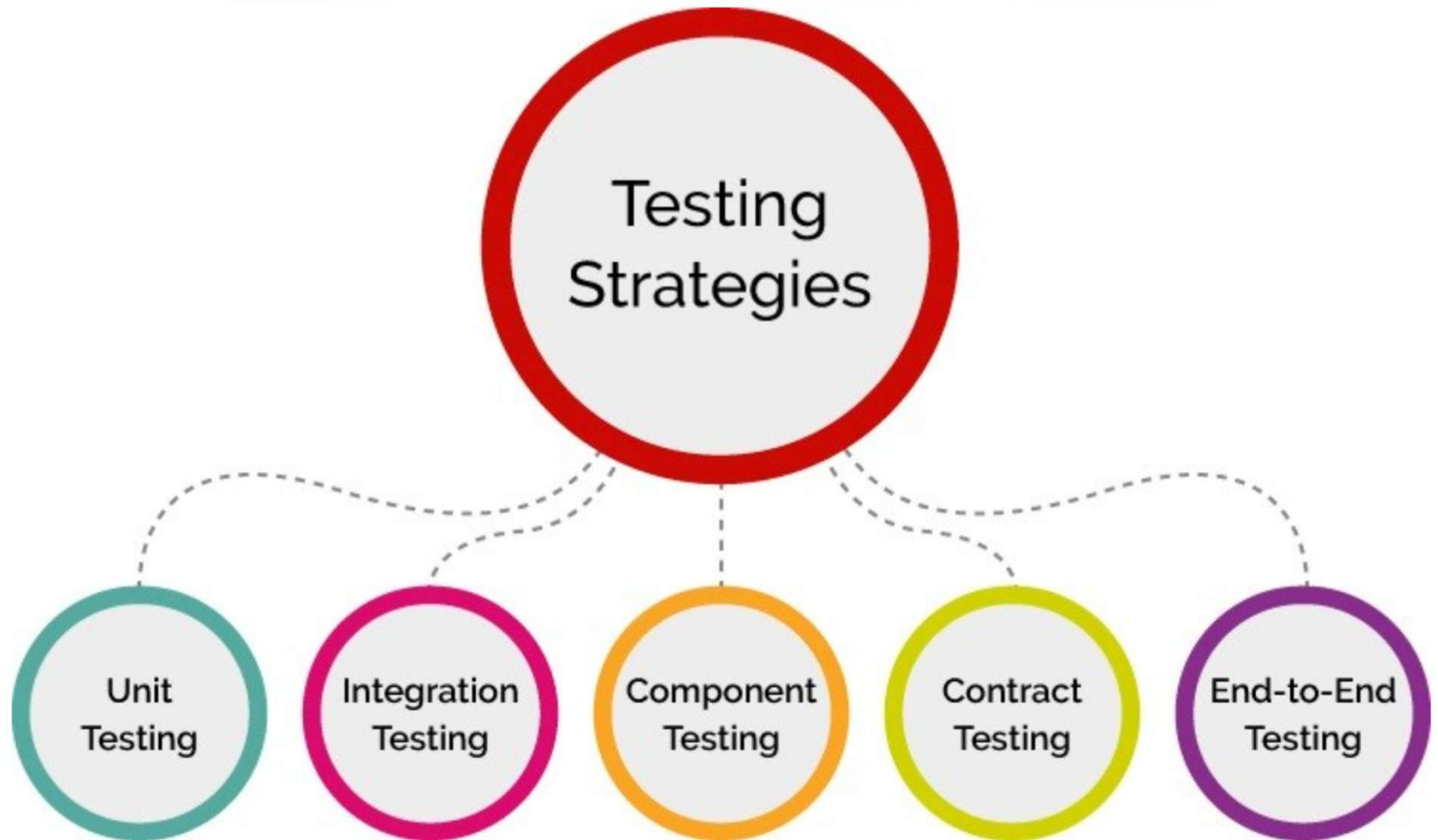
4. Testes de contrato de serviço:



ObterPacientePeloNome

Validação
O consumidor fez a requisição esperada?
O consumidor tratou de maneira correta a resposta esperada?
O produtor tratou a requisição esperada?
O provedor retornou a resposta esperada?
O provedor armazenou corretamente os dados da requisição?

Construção - Teste

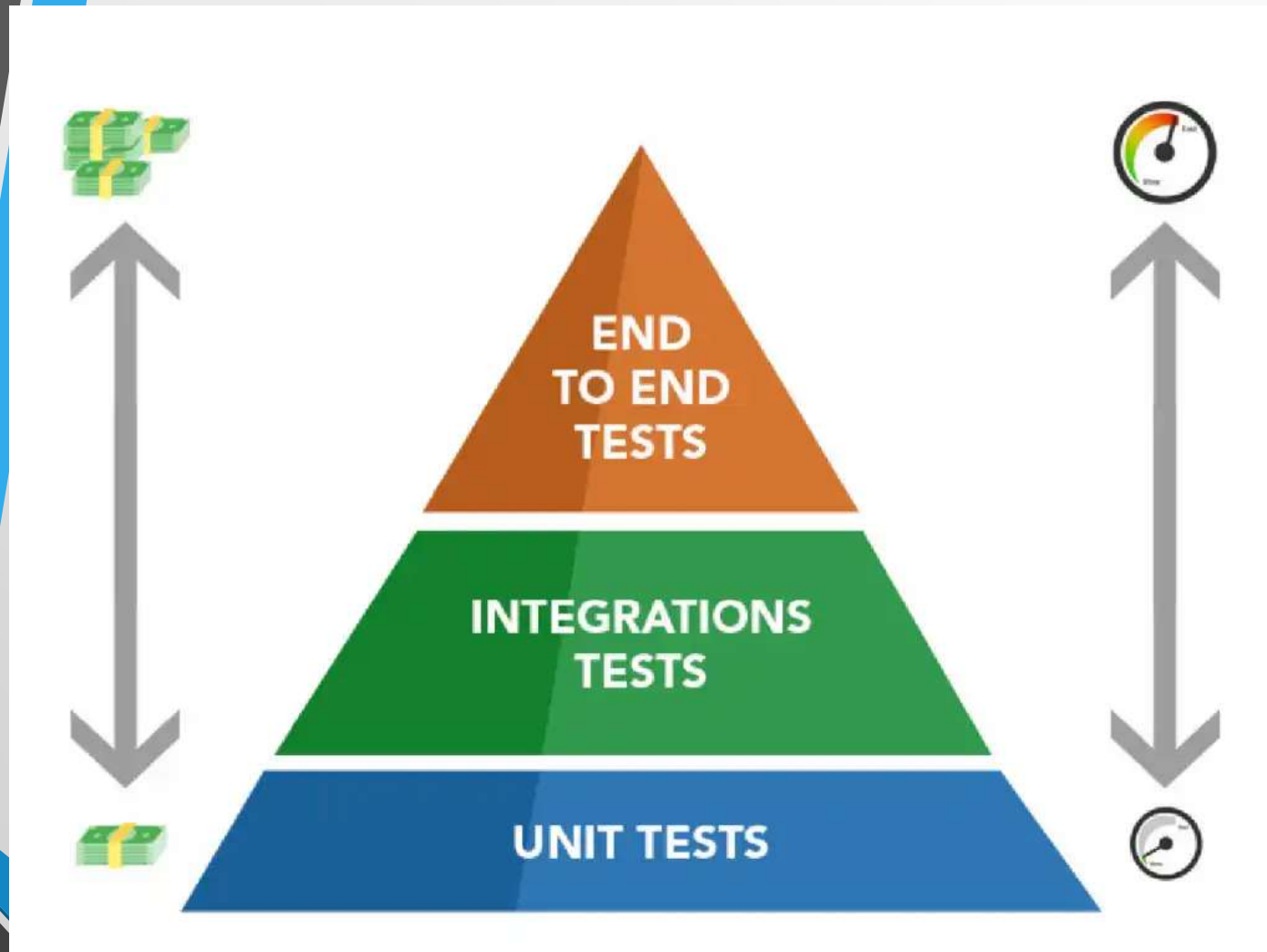


Construção - Teste

5. Teste End-to-End

- Utilizados para testar o fluxo completo de um sistema de serviços.
- Verifica se todos os critérios de aceitação estão cobertos e contemplados por todos os serviços, buscando testar o sistema por inteiro, de ponta-a-ponta.
- Testam jornadas completas de usuários.
 - ✓ Ex: Testar o agendamento de uma consulta médica

Construção - Teste



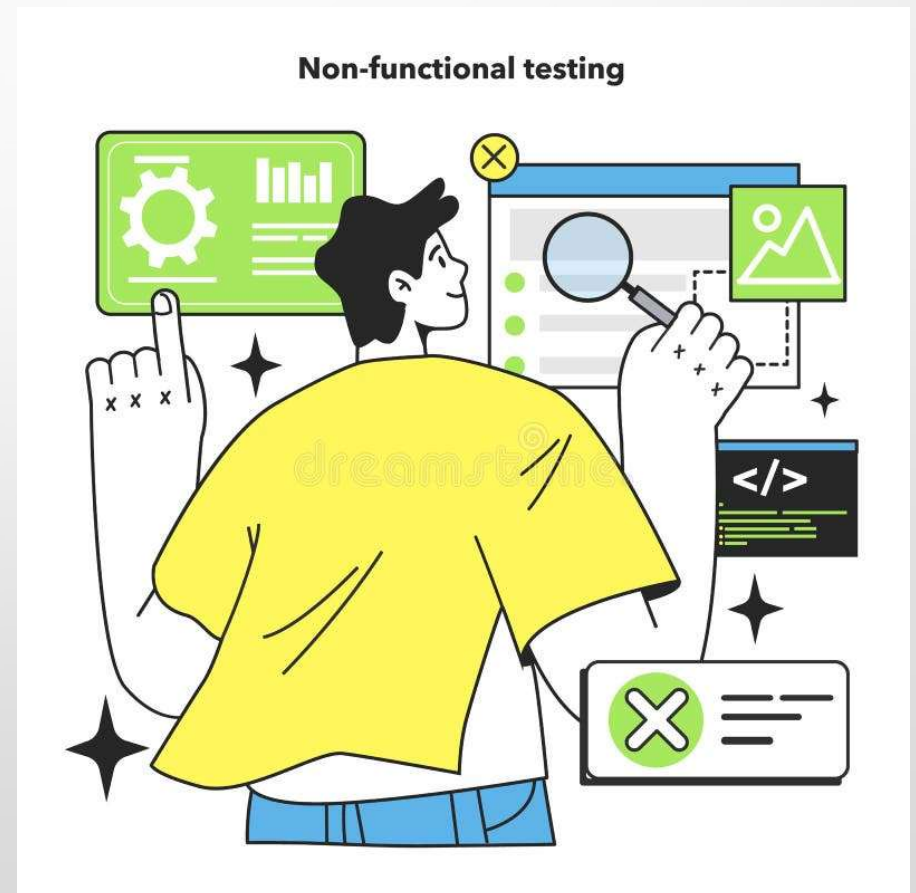
- Automação: parcial
- Manual: importante
- Dados: reais
- Mock/Stub: evitar
- Foco: experiência do usuário

- Automação: maioria
- Manual: pouco
- Dados: misto
- Mock/Stub: moderado
- Foco: comunicação entre serviços

- Automação: total
- Dados: simulados
- Mock/Stub: intenso
- Foco: lógica isolada

Construção - Teste

Testes não Funcionais



Construção - Teste

1. Testes de Segurança

Objetivo

- Garantir que o serviço protege dados e acessos.

O que testar

- ✓ autenticação
- ✓ autorização
- ✓ proteção de dados sensíveis
- ✓ ataques (injeção, acesso indevido)

Construção - Teste

2. Testes de Desempenho

Objetivo

Medir o desempenho do serviço em termos de:

- ✓ tempo de resposta
- ✓ throughput (requisições por segundo)
- ✓ tempo de processamento de operações

Exemplo

- 1 usuário envia requisição
- Resposta deve vir em até **2 segundos**
- Foco: **velocidade**

Construção - Teste

3. Testes de Carga

Objetivo

Avaliar o comportamento do sistema com muitos usuários simultâneos.

Exemplo

- 500 usuários criando pedidos ao mesmo tempo
- Verificar:
 - ✓ se o sistema aguenta
 - ✓ se não cai
 - ✓ se não degrada muito

Foco: **capacidade**

Construção - Teste

4. Testes de Estresse

Objetivo

Identificar o limite do sistema e como ele se comporta em condições extremas.

O que testar

- ✓ ponto de falha
- ✓ comportamento sob sobrecarga
- ✓ recuperação após falha

Exemplo

- Aumentar carga até 2000 requisições simultâneas
observar quando o serviço para de responder

Construção - Teste

5. Testes de Disponibilidade

Objetivo

- Verificar se o serviço permanece disponível ao longo do tempo.

O que testar

- ✓ uptime (tempo ativo)
- ✓ tolerância a falhas

Exemplo

- simular queda do serviço de pagamento
- verificar se o sistema pedido continua operando parcialmente

Construção - Teste

Tipo de Teste: Teste Unitário

Objetivo do Teste	Validar a lógica das operações do serviço de cliente de forma isolada, garantindo o correto funcionamento das funcionalidades.
Técnica	Testes unitários com isolamento total das dependências, utilizando mocks para o repositório de dados e serviços externos. ✓ Execução direta dos métodos da classe de negócio ✓ Uso de dados simulados ✓ Sem acesso a banco ou outros serviços
Critério de Finalização	• Todas as operações testadas individualmente • Casos positivos e negativos cobertos • Execução dos testes sem falhas

Construção - Teste

Tipo de Teste: Teste de Integração

Objetivo do Teste	Verificar se o Serviço Cliente se comunica corretamente com o Serviço de Consulta de Crédito (Serasa), garantindo que a requisição seja enviada corretamente, a resposta seja correta.
Técnica	▪ Executar chamadas (reais/simuladas) entre os serviços. ▪ Validar o contrato de comunicação (mensagens JSON) ▪ Simular diferentes respostas do serviço externo (quando necessário)
Critério de Finalização	Teste será considerado concluído quando: • Todas as requisições forem enviadas corretamente ao serviço de crédito • As respostas forem: ✓ recebidas sem erro de comunicação ✓ interpretadas corretamente pelo sistema • Todos os cenários forem executados com sucesso • Falhas externas forem tratadas adequadamente